HiTicket — IT Helpdesk & Ticketing System

Project Report

HiTicket

AI-Assisted IT Helpdesk and Ticketing Platform

A Full-Stack Web Application

Submitted in partial fulfillment of the requirements for the award of the degree

Department of Computer Science & Engineering / Information Technology

Submitted by

Chaitanya Deep C

Academic Year: 2025–2026

Live Application: https://hiticket.vercel.app

Declaration

I hereby declare that the project entitled **"HiTicket — AI-Assisted IT Helpdesk and Ticketing Platform"** submitted by me is a record of original work carried out independently. The project has not been submitted elsewhere for the award of any degree or diploma. All information sourced from books, journals, open-source libraries, and electronic media has been duly acknowledged with references.

Signature: _____

Date: March 2026

Acknowledgements

I express my sincere gratitude to my project guide and faculty members for their consistent guidance, encouragement, and constructive feedback throughout the development of this project. I also acknowledge the open-source community whose libraries and tools formed the foundation of this application — including the contributors behind React, Express.js, MongoDB, Tailwind CSS, and all supporting packages. Special thanks to Vercel and Render for providing accessible production-grade hosting infrastructure that enabled a complete end-to-end cloud deployment.

Abstract

HiTicket is a production-grade, full-stack IT helpdesk and ticketing platform designed to streamline the lifecycle of IT support requests within an organization. The system enables end-users to raise support tickets through a conversational, AI-guided chatbot interface, eliminating the complexity and friction typically associated with traditional IT ticketing portals. Agents and administrators manage their workloads from a unified dashboard equipped with real-time KPIs, SLA breach tracking, ticket aging analysis, knowledge base management, and role-based access control.

The application is built on a modern technology stack: a **React 19** single-page application (SPA) on the frontend, a **Node.js + Express** REST API on the backend, and **MongoDB Atlas** as the primary database. The entire solution is deployed on cloud infrastructure — Vercel for the frontend and Render for the backend — with **MongoDB Atlas**, **Cloudinary**, and the **Gmail REST API** as managed external services.

Key capabilities include: two-factor authentication (TOTP and Email OTP), JWT-based session management with token versioning, automated round-robin ticket assignment, a knowledge base with deflection built into the ticket creation flow, Cloudinary-powered file attachments, scheduled weekly email digests via `node-cron`, full OWASP Top 10 security hardening, and Progressive Web App (PWA) support for offline capability and mobile install.

The project demonstrates practical application of full-stack web development principles including RESTful API design, database modelling, client-side state management, role-based authorization, secure authentication patterns, and CI/CD-based cloud deployment.

Keywords: IT Helpdesk, Ticketing System, React, Node.js, MongoDB, REST API, JWT, Two-Factor Authentication, PWA, Role-Based Access Control

---

Table of Contents

1. Introduction

- 1.1 Problem Statement
- 1.2 Project Objectives
- 1.3 Project Scope
- 1.4 Organisation of the Report

2. Literature Survey

- 2.1 Existing Systems
- 2.2 Limitations of Existing Systems
- 2.3 Proposed Solution

3. System Requirements

- 3.1 Functional Requirements
- 3.2 Non-Functional Requirements
- 3.3 Hardware Requirements
- 3.4 Software Requirements

4. System Architecture & Design

- 4.1 High-Level Architecture
- 4.2 Three-Tier Architecture
- 4.3 Frontend Architecture
- 4.4 Backend Architecture
- 4.5 Data Flow

5. Technology Stack

- 5.1 Frontend Technologies
- 5.2 Backend Technologies
- 5.3 External Services
- 5.4 Development Tools

6. Database Design

- 6.1 Data Model Overview
- 6.2 User Schema
- 6.3 Ticket Schema
- 6.4 KbArticle Schema
- 6.5 Schema Relationships

7. System Modules & Implementation

- 7.1 Authentication Module
- 7.2 Ticket Management Module
- 7.3 AI Chatbot (Ticket Creation Wizard)
- 7.4 Knowledge Base Module
- 7.5 Admin Dashboard & Analytics
- 7.6 User & Role Management
- 7.7 Email Notification System
- 7.8 File Attachment System

- 7.9 Progressive Web App (PWA)
- 7.10 UI / Design System

8. Security Implementation

- 8.1 Authentication Security
- 8.2 Authorization & Access Control
- 8.3 OWASP Top 10 Mapping
- 8.4 Rate Limiting
- 8.5 Data Sanitization

9. Testing

- 9.1 Testing Approach
- 9.2 Functional Test Cases
- 9.3 Security Test Scenarios
- 9.4 Build Validation

10. Deployment

- 10.1 Deployment Architecture
- 10.2 Frontend — Vercel
- 10.3 Backend — Render
- 10.4 Database — MongoDB Atlas
- 10.5 Environment Variables
- 10.6 CI/CD Pipeline

11. Results & Discussion

- 11.1 Achieved Outcomes
- 11.2 UI Walkthrough
- 11.3 Performance Observations

12. Conclusion & Future Scope

- 12.1 Conclusion
- 12.2 Future Enhancements

13. References

14. Appendix

- A. API Endpoint Reference
- B. Keyboard Shortcuts
- C. Chatbot Slash Commands

Chapter 1: Introduction

1.1 Problem Statement

In any organization with an IT department, employees encounter technical issues that need resolution — hardware failures, software crashes, network connectivity problems, password resets, and access request. Currently, IT support in many organizations is managed through informal channels such as emails, phone calls, or walk-ins. This leads to:

- **No structured tracking** — Issues are forgotten, duplicated, or never followed up.
- **No accountability** — No clear record of who is working on what, or how long issues take to resolve.
- **No prioritization** — Critical issues receive the same treatment as low-priority requests.
- **No self-service** — Users cannot look up known solutions before reaching out to the IT team.
- **No visibility into performance** — Management cannot assess the IT department's efficiency or workload.

Traditional IT Service Management (ITSM) tools like ServiceNow and Jira Service Management exist, but they are enterprise-grade, expensive, overly complex for small to mid-sized organizations, and require significant training to operate.

There is a clear need for a lightweight, modern, easy-to-use helpdesk platform that provides ticket management, knowledge base, analytics, and role-based workflows — without the overhead of enterprise ITSM software.

1.2 Project Objectives

The primary objectives of the HiTicket project are:

1. Build a complete **IT helpdesk platform** deployable as a cloud web application accessible from any device.

2. Enable users to **raise support tickets in under 60 seconds** through a guided conversational chatbot, without requiring any training.

3. Provide agents and administrators with a **unified dashboard** covering all ticket management operations, SLA tracking, and workload analytics.

4. Implement a **knowledge base** integrated directly into the ticket creation flow to reduce unnecessary ticket submissions.

5. Enforce **role-based access control** with three distinct roles (User, Agent, Admin) — each with appropriately scoped capabilities.

6. Apply **enterprise-grade security** aligned with OWASP Top 10 — including two-factor authentication, JWT session management, rate limiting, and NoSQL injection prevention.

7. Support **email notifications** at every ticket lifecycle stage via Gmail REST API.

8. Deliver a **Progressive Web App** — installable, offline-capable, and responsive across mobile and desktop.

1.3 Project Scope

In Scope:

- Web-based ticketing and helpdesk platform accessible at https://hiticket.vercel.app
- Complete ticket lifecycle: creation → triage → in progress → resolved → closed
- Three user roles: User, Agent, Admin
- Two-factor authentication (TOTP + Email OTP)
- Knowledge base with full-text search and helpfulness ratings
- Admin analytics dashboard with charts, SLA breach tracking, and ticket aging analysis
- File attachments stored on Cloudinary CDN
- Email notifications via Gmail REST API
- Weekly automated digest emails via scheduled cron job

- Progressive Web App (installable, offline support)
- RESTful JSON API backend

Out of Scope:

- Native mobile application (iOS/Android packaging)
- Real-time WebSocket communication (polling used instead)
- Single Sign-On (SSO) with Google/Microsoft OAuth
- Multi-tenant support with department isolation
- SLA auto-escalation workflows

1.4 Organisation of the Report

This report is organized into twelve chapters. Chapter 2 surveys existing systems and the motivation for this project. Chapter 3 defines the system requirements. Chapter 4 presents the architecture and design. Chapter 5 covers the technology stack. Chapter 6 describes the database design. Chapter 7 explains each system module and its implementation. Chapter 8 covers the security implementation. Chapter 9 describes the testing process. Chapter 10 documents deployment. Chapter 11 presents results and observations. Chapter 12 draws conclusions and outlines future scope.

Chapter 2: Literature Survey

2.1 Existing Systems

Several IT helpdesk and ticketing systems exist in the market:

System	Type	Key Features	Pricing
ServiceNow	Enterprise ITSM	Full ITIL workflows, CMDB, change management	$100+/user/month
Jira Service Management	Project-integrated ITSM	Kanban, SLA, integrates with Jira/Confluence	$20+/agent/month
Zendesk	Customer support	Omnichannel, AI suggestions, extensive integrations	$55+/agent/month
Freshdesk	SMB helpdesk	Email ticketing, basic SLA, knowledge base	Free tier + paid
osTicket	Open source	Email-to-ticket, basic dashboard	Free/self-hosted
GLPI	Open source ITSM	Asset management, ticketing, reporting	Free/self-hosted

---

2.2 Limitations of Existing Systems

1. **High cost** — Enterprise tools like ServiceNow and Jira Service Management are priced beyond what most educational institutions and small businesses can afford.

2. **Steep learning curve** — Most tools require several hours of configuration and training before an agent can handle tickets productively.

3. **No guided ticket creation** — Users are presented with long forms requiring technical knowledge (category IDs, SLA policies, components) that non-IT staff cannot fill meaningfully.

4. **No integrated AI guidance** — Ticket creation is form-based, not conversational. Incorrect categorization is common.

5. **Limited customization for small teams** — Open source options like osTicket are functional but lack modern UX, mobile support, and PWA capability.

6. **No mobile-first PWA** — Most tools have mobile apps as paid add-ons; few offer offline capability.

2.3 Proposed Solution

HiTicket addresses these gaps by providing:

- A **free, open, cloud-deployed** platform with no per-user pricing
- A **conversational chatbot-style ticket creation** flow that guides even non-technical users
- **Knowledge base deflection** built into the creation flow, surfacing solutions before a ticket is submitted
- A **clean, modern dark UI** with responsive design and PWA capability
- **Full OWASP security hardening** competitive with commercial tools
- **One-click cloud deployment** on Vercel and Render with minimal configuration

---

Chapter 3: System Requirements

3.1 Functional Requirements

User Requirements:

- FR-01: Users shall be able to register with email and password, and log in to access the platform.
- FR-02: Users shall be able to raise a ticket through a guided 4-step chatbot wizard.
- FR-03: Users shall be able to view, filter, sort, and search their own tickets.
- FR-04: Users shall be able to add comments to their tickets and upload file attachments.
- FR-05: Users shall receive email notifications at each stage of their ticket's lifecycle.
- FR-06: Users shall be able to mark themselves as watchers on any ticket.
- FR-07: Users shall be able to enable two-factor authentication via TOTP or Email OTP.
- FR-08: Users shall be able to submit a post-resolution satisfaction (CSAT) rating.
- FR-09: Users shall be able to access a public Knowledge Base for self-service resolution.

Agent Requirements:

- FR-10: Agents shall be able to view and manage all tickets across all users.
- FR-11: Agents shall be able to update ticket status, priority, and assignment.
- FR-12: Agents shall be able to add internal notes visible only to staff.
- FR-13: Agents shall be able to assign due dates to tickets.
- FR-14: Agents shall be able to view SLA breach status per ticket.

Admin Requirements:

- FR-15: Admins shall be able to view an analytics dashboard with KPI cards, charts, and leaderboards.
- FR-16: Admins shall be able to filter dashboard data by date range (7d / 30d / 90d / All).
- FR-17: Admins shall be able to manage all users — assign roles, activate or deactivate accounts.
- FR-18: Admins shall be able to perform bulk status updates and bulk deletion of tickets.
- FR-19: Admins shall be able to view an activity audit log of all admin actions.
- FR-20: Admins shall be able to view post-resolution survey results.

System Requirements:

- FR-21: The system shall auto-assign newly created tickets to the least-loaded active agent (round-robin).
- FR-22: The system shall send weekly digest emails to opted-in users every Monday at 08:00 UTC.
- FR-23: The system shall maintain a full audit history of all field changes on every ticket.
- FR-24: The system shall function as a Progressive Web App — installable on mobile and desktop.

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API responses shall complete within 500ms under normal load
NFR-02	Availability	System shall be accessible 24/7 with Vercel/Render auto-recovery
NFR-03	Security	All passwords shall be hashed using bcrypt (cost factor 12)
NFR-04	Security	All API communication shall occur over HTTPS
NFR-05	Security	OWASP Top 10 risks shall be mitigated (detailed in Chapter 8)
NFR-06	Scalability	Backend shall be stateless — horizontally scalable without session stores
NFR-07	Usability	Ticket creation shall require no technical knowledge; complete in under 60 seconds
NFR-08	Compatibility	Application shall function on Chrome, Firefox, Edge, Safari — desktop and mobile
NFR-09	Offline	PWA shall serve cached data when the device is offline
NFR-10	Maintainability	Frontend and backend shall be independently deployable separate services

---

3.3 Hardware Requirements

Development:

- CPU: 2+ cores (any modern processor)
- RAM: 8 GB minimum
- Storage: 10 GB free disk space
- Internet: Broadband connection for cloud service access

Production (cloud-hosted — no dedicated hardware required):

- Frontend: Vercel CDN (globally distributed edge network)
- Backend: Render Web Service (shared container, 512 MB RAM)
- Database: MongoDB Atlas M0 shared cluster

3.4 Software Requirements

Development Environment:

- Node.js 18+
- npm 9+
- Git
- VS Code (or any modern code editor)
- MongoDB Compass (optional, for local DB inspection)

Production Environment:

- Vercel (frontend hosting)
- Render (backend hosting)
- MongoDB Atlas (database)
- Cloudinary account (file storage)
- Google Cloud OAuth2 credentials (email via Gmail API)

Chapter 4: System Architecture & Design

4.1 High-Level Architecture

HiTicket is a **decoupled three-tier web application** with clear separation between the presentation, application, and data layers. The frontend and backend are entirely

independent services — they communicate exclusively through a JSON REST API over HTTPS.

```
┌──────────────────────────────────────────────────┐
│  CLIENT TIER                                     │
│  Browser / PWA (React 19 SPA)                    │
│  Hosted: Vercel CDN                              │
│  URL: https://hiticket.vercel.app                │
└──────────────────────┬───────────────────────────┘
                       │  HTTPS REST (Bearer JWT)
                       ▼
┌──────────────────────────────────────────────────┐
│  APPLICATION TIER                                │
│  Node.js 20 + Express 4                          │
│  Hosted: Render Web Service                      │
└──────┬──────────────────────────────┬────────────┘
       │ Mongoose (TLS)               │ OAuth2 / HTTPS
       ▼                              ▼
┌──────────────────┐        ┌────────────────────────┐
│  MongoDB Atlas   │        │  Gmail REST API        │
│  (Shared Cluster)│        │  Cloudinary CDN        │
└──────────────────┘        └────────────────────────┘
```

4.2 Three-Tier Architecture

Presentation Tier (Frontend):

- Built with React 19 as a Single-Page Application (SPA)
- All routing handled client-side using React Router v7
- UI built entirely in Tailwind CSS v3 with no external component library
- State managed using React Context API (AuthContext, ThemeContext, ToastContext)
- Communicates with the backend exclusively via Axios HTTP requests

Application Tier (Backend):

- Node.js 20 + Express 4 RESTful API
- Stateless design — no server-side sessions; authentication via JWT tokens
- Middleware chain: Helmet → Rate Limiter → CORS → Body Parser → Mongo Sanitize → Route Handler → Error Handler
- Business logic organized into route files, middleware, models, and utility functions

Data Tier:

- MongoDB Atlas — cloud-hosted document database
- Mongoose ODM for schema definition, validation, and query abstraction
- Three primary collections: `users`, `tickets`, `kbarticles`

4.3 Frontend Architecture

The frontend source is organized under `helpdesk-ai/src/`:

```
src/
├── App.jsx            # Router root — all routes, 3 guards, global key handlers
├── api/api.js         # Axios instance — base URL + auth interceptor
├── context/           # AuthContext, ThemeContext, ToastContext
├── hooks/             # useInactivityLogout, useScrollHide
├── pages/             # 15 page components
└── components/
    ├── layout/        # Navbar, BottomNav, Breadcrumb, PageWrapper, ScrollToTop
    ├── ui/            # StatCard, Badge, Card, Modal, Avatar
    └── (root)         # CommandPalette, ChatBubble, OTPInput, TicketCard, Onboardin
```

Key design patterns used:

- **Context API** for global state (auth, theme, toasts)
- **Custom hooks** for reusable logic (inactivity logout, scroll hide)
- **Route guards** (`ProtectedRoute`, `StaffRoute`, `AdminRoute`) wrapping sensitive pages
- **Axios interceptor** for automatic JWT attachment on every request and 401 redirect handling

- **Component composition** over inheritance — no class components

---

4.4 Backend Architecture

The backend source is organized under `helpdesk-api/`:

```
helpdesk-api/
├── server.js           # Express bootstrap, middleware chain, route mounting, cron
├── middleware/auth.js # protect · agentOrAdmin · adminOnly
├── models/             # User.js · Ticket.js · KbArticle.js
├── routes/             # auth · tickets · users · kb · logs · feedback
└── utils/
    ├── email.js        # HTML email templates + Gmail API send functions
    └── storage.js      # Cloudinary upload/delete helpers
```

Middleware chain (every request):

1. `helmet()` — applies 12 HTTP security headers
2. `trust proxy: 1` — accurate client IP behind Render's load balancer
3. `globalLimiter` — IP-based rate limiting applied to all routes
4. `cors(whitelist)` — restricts origins to the explicit production URL (`CLIENT_URL`) and local development ports only
5. `express.json({ limit: '50kb' })` — parses JSON body, caps size
6. `mongoSanitize()` — strips `$` and `.` to prevent NoSQL injection
7. Route handler
8. Global error handler — returns `{ error: message }`

4.5 Data Flow

Standard authenticated request flow:

1. User performs an action in the React UI (e.g., updating ticket status)
2. React component calls an Axios function — the interceptor appends the JWT: `Authorization: Bearer`
3. HTTPS request reaches Render (Express)
4. Middleware chain validates JWT, checks tokenVersion against the database
5. Route handler performs business logic (read/write to MongoDB Atlas via Mongoose)
6. JSON response returned — React updates state, UI re-renders
7. Side effects (emails) fired asynchronously after response — never block the response

---

Chapter 5: Technology Stack

5.1 Frontend Technologies

Technology	Version	Purpose
React	19.x	UI component framework — declarative, component-based
Vite	8.x	Build tool and development server — 10x faster than CRA
React Router	v7	Client-side routing with nested routes and guards
Tailwind CSS	v3	Utility-first CSS framework — entire UI built without external component libs
Recharts	2.x	SVG-based charting library for analytics dashboard
Axios	1.x	HTTP client with request/response interceptors
vite-plugin-pwa	1.x	PWA manifest + Workbox service worker integration

Why React over alternatives:

React 19's Concurrent Features (automatic batching, Suspense boundaries) improve perceived performance for data-heavy views like the admin dashboard. React Router v7's nested route support cleanly models the admin sub-pages (`/admin`, `/admin/users`, `/admin/logs`).

Why Vite over CRA:

Vite uses native ES Modules during development, giving sub-100ms hot module replacement versus CRA's webpack-based seconds-long rebuilds. Production builds are also significantly smaller.

5.2 Backend Technologies

Technology	Version	Purpose
Node.js	20.x	JavaScript runtime — event-driven, non-blocking I/O
Express	4.x	Minimal web framework — routing, middleware, error handling
Mongoose	8.x	MongoDB ODM — schema validation, query building, hooks
jsonwebtoken	9.x	JWT signing and verification (HS256)
bcryptjs	2.x	Password hashing (high cost factor, intentionally slow)
speakeasy	2.x	TOTP generation and verification (RFC 6238)
qrcode	1.x	QR code generation for TOTP enrollment
node-cron	3.x	Cron-style scheduled task execution (weekly digest)
multer	1.x	Multipart form data parsing for file uploads (memory storage)
helmet	7.x	HTTP security headers (12 headers)
express-rate-limit	7.x	IP-based request rate limiting
express-mongo-sanitize	2.x	NoSQL injection prevention
cors	2.x	Cross-origin resource sharing configuration

5.3 External Services

Service	Purpose	Integration Method
MongoDB Atlas	Cloud-hosted document database	Mongoose ODM over TLS connection string
Cloudinary	File/image CDN and storage	`upload_stream` API via `cloudinary` npm package
Gmail REST API	Transactional email sending	`googleapis` npm package with OAuth2 credentials
Vercel	Frontend static hosting and CDN	Auto-deploy from Git on push to main
Render	Backend web service hosting	Auto-deploy from Git on push to main

Why Gmail REST API instead of SMTP/Nodemailer?

Render's free tier blocks outbound SMTP ports 465 and 587. The Gmail REST API communicates over HTTPS port 443, which is never blocked. This was a deliberate architectural decision to ensure email delivery in the production environment.

5.4 Development Tools

- **Git** — version control; monorepo structure with `helpdesk-ai/` and `helpdesk-api/` as subdirectories
- **VS Code** — primary editor with ESLint and Tailwind IntelliSense
- **MongoDB Compass** — GUI for inspecting the Atlas database during development
- **Postman** — API testing during backend development
- **npm** — package manager; `.npmrc` with `legacy-peer-deps=true` for Vercel PWA plugin compatibility

---

Chapter 6: Database Design

6.1 Data Model Overview

HiTicket uses a **document-oriented data model** in MongoDB. The document model was chosen because ticket objects naturally contain nested arrays (comments, notes, history entries, attachments) that would require multiple relational table joins in SQL, but map naturally to a single BSON document.

Three primary collections:

1. **users** — Authentication identity, role, notification preferences, 2FA configuration
2. **tickets** — The core entity — full lifecycle, sub-documents, audit trail
3. **kbarticles** — Knowledge base content with view and helpfulness counters

---

6.2 User Schema

```
users collection:
{
  name:       String  (required, trimmed)
  email:      String  (required, unique, lowercase)
  password:   String  (bcrypt-hashed, select: false — never returned in API)
  role:       Enum['user', 'agent', 'admin']  (default: 'user')
  department: String
  phone:      String
  jobTitle:   String
  location:   String
  isActive:   Boolean (default: true)
  tokenVersion: Number (default: 0 — incremented on login/logout for session invalida

  notificationPrefs: {
    emailEnabled:  Boolean (default: true)
    ticketUpdates: Boolean (default: true)
    newComments:   Boolean (default: true)
    weeklyDigest:  Boolean (default: false)
  }

  twoFactor: {
    enabled:          Boolean (default: false)
    method:           Enum['email', 'totp']
    totpSecret:       String (select: false)
    pendingOtp:       String (select: false)
    pendingOtpExpiry: Date   (select: false)
  }

  createdAt, updatedAt (Mongoose timestamps)
}
```

Hooks: `pre('save')` — if `password` field is modified, re-hashes it with `bcrypt.hash(password, 12)`.

Methods: `matchPassword(entered)` — performs timing-safe `bcrypt.compare()` and returns a boolean.

6.3 Ticket Schema

```
tickets collection:
{
  ticketId:    String  (unique, auto-generated: TKT-0001, TKT-0002 ...)
  title:       String  (required, trimmed)
  description: String  (required)
  category:    String  (required — Hardware/Software/Network/Access/Email/Printer/VPI
  subType:     String  (sub-category within category)
  priority:    Enum['Low', 'Medium', 'High', 'Critical']  (default: 'Medium')
  status:      Enum['Open', 'In Progress', 'Resolved', 'Closed']  (default: 'Open')
  createdBy:   ObjectId → User  (required, populated)
  assignedTo:  String  (agent display name — set automatically on creation)
  dueDate:     Date    (null by default; set by agents/admins)
  watchers:    [ObjectId → User]  (users watching this ticket)

  comments: [{
    text, author (ObjectId), authorName, createdAt, updatedAt
  }]

  internalNotes: [{
    text, author (ObjectId), authorName, authorRole, createdAt, updatedAt
    — NEVER returned in API responses to non-staff users
  }]

  history: [{
    action, field, from, to, by (ObjectId), byName, createdAt
    — Full audit trail of every change
  }]

  attachments: [{
    url (Cloudinary secure_url), publicId, filename, mimetype, size,
    uploadedBy (ObjectId), uploaderName, createdAt
  }]

  satisfaction: {
    rating (Number 1-5), feedback (String), submittedAt (Date)
  }
```

```
  resolvedAt: Date  (auto-set when status changes to 'Resolved')
  createdAt, updatedAt (Mongoose timestamps)
}
```

Hooks: `pre('save')` — if document is new, auto-generates TKT-NNNN ticketId (finds last, increments, pads to 4 digits) and pushes initial "Ticket created" history entry. If status changes to 'Resolved', sets `resolvedAt`.

6.4 KbArticle Schema

```
kbarticles collection:
{
  title:       String  (required, trimmed)
  content:     String  (required — Markdown text)
  category:    String  (default: 'General')
  tags:        [String]
  author:      ObjectId → User
  authorName:  String
  isPublished: Boolean (default: true)
  views:       Number  (incremented on GET /:id)
  helpful:     Number  (incremented by thumb-up votes)
  notHelpful:  Number  (incremented by thumb-down votes)
  createdAt, updatedAt
}
```

6.5 Schema Relationships

```
User (1) ──────────< Ticket (Many)      [via createdBy]
User (1) ──────────< Ticket (Many)      [via watchers array]
User (1) ──────────< Comment (Many)     [via comments[].author]
User (1) ──────────< KbArticle (Many)   [via author]
Ticket (1) ────────< Comment (Many)     [embedded sub-document]
Ticket (1) ────────< InternalNote (Many) [embedded sub-document]
Ticket (1) ────────< History (Many)     [embedded sub-document]
Ticket (1) ────────< Attachment (Many) [embedded sub-document]
```

Comments, internal notes, history entries, and attachments are stored as **embedded sub-documents** within the ticket document. This eliminates multi-collection joins for the most common read operation (fetching a complete ticket with all its details).

Chapter 7: System Modules & Implementation

7.1 Authentication Module

Authentication is the foundation of HiTicket's security model.

Registration flow:

1. User submits `{ name, email, password }` via `POST /api/auth/register`
2. Email uniqueness validated against the `users` collection
3. `User.create()` triggers the pre-save hook — `bcrypt.hash(password, 12)` replaces plaintext
4. A long-lived JWT is signed with `HS256` using the server's `JWT_SECRET`
5. Response: `{ token, user: { id, name, email, role } }`

Login flow:

1. `bcrypt.compare(entered, stored)` — timing-safe comparison
2. If `user.isActive === false` → HTTP 403 Forbidden

3. If 2FA is enabled: a short-lived `tempToken` (typed to reject against protected routes) is issued; the full JWT is NOT given yet

4. If 2FA is not enabled: `tokenVersion++`, full JWT issued

JWT Token Architecture:

The payload contains the user ID, a `tokenVersion` counter for server-side invalidation, and standard expiry claims. `tokenVersion` is a critical security field — every login increments it. The `protect` middleware checks that `decoded.tokenVersion === user.tokenVersion`. This means logging out (which increments `tokenVersion`) invalidates all previously issued tokens from all devices.

Two-Factor Authentication (2FA):

Method	Library	Flow
Email OTP	Custom (crypto.randomInt)	Time-limited numeric OTP, sent via Gmail API; verified by constant-time string comparison
TOTP	speakeasy (RFC 6238)	Secret generated server-side; QR code via `qrcode`; user scans with Authenticator app; verified with RFC 6238 clock drift tolerance

After successful 2FA verification: `tokenVersion++` and the full JWT is issued.

7.2 Ticket Management Module

Ticket creation with auto-assignment:

1. Backend calls `getNextAgent()` — fetches all active agents, counts their currently open tickets, picks the one with the fewest (least-loaded round-robin)

2. `Ticket.create()` triggers the pre-save hook: auto-generates ticketId in TKT-NNNN format; pushes "Ticket created" to history

3. A retry loop (3 attempts) handles rare `ticketId` collisions under concurrent creation (MongoDB error code 11000)

4. `sendTicketCreated()` is called asynchronously — never blocks the response

Ticket lifecycle transitions:

```
Open ──→ In Progress ──→ Resolved ──→ Closed
  ↑                          |
  └──────────────────────────┘   (reopen — users only)
```

SLA thresholds (enforced in Admin Dashboard — SLA Breach tab):

Priority	SLA (hours)
Critical	4
High	8
Medium	24
Low	72

Watcher feature: Any authenticated user can toggle watch/unwatch on any ticket. The watcher's `ObjectId` is pushed to/pulled from `ticket.watchers[]`. The localStorage `userId` is used to reconcile the watch state on the frontend.

7.3 AI Chatbot (Ticket Creation Wizard)

The chatbot is a **state-machine-based conversational UI**, not a large language model. Its intelligence is deterministic and keyword-based, making it predictable and reliable without external API calls.

4-step wizard flow:

1. **Category** — 8 tile grid (Hardware, Software, Network, Access, Email, Printer, VPN, Other) or free text

2. **Sub-type** — dynamic chip list that changes based on the selected category

3. **Details + KB Deflection** — category-specific follow-up questions; simultaneously queries `/api/kb?q=` and shows up to 3 matching articles as dismissible chips

4. **Confirm + Submit** — shows auto-detected priority; user can adjust before final submission

Auto-categorization (keyword regex):

```
const CATEGORY_KEYWORDS = {
  Hardware:  /laptop|computer|mouse|keyboard|monitor|printer|hardware|device|screen/i
  Software:  /software|app|install|crash|error|update|virus/i,
  Network:   /internet|wifi|network|vpn|connection|ethernet|slow|dns/i,
  Access:    /password|login|account|access|permission|locked|reset/i,
  // ...
}
```

Priority auto-detection:

```
if (/urgent|critical|emergency|not working|broken|down|outage/i.test(description))
  priority = 'Critical';
else if (/important|failing|cannot|blocked/i.test(description))
  priority = 'High';
else priority = 'Medium';
```

6 pre-built ticket templates (accessible via `/template` slash command): Reset Password, VPN Access, New Laptop Setup, Printer Issue, Email Access, Software Access.

Slash commands: `/status` (inline ticket lookup), `/agent` (agent handoff request), `/template` (template grid), `/kb` (KB search), `/clear` (reset conversation).

KB Deflection step: Before the confirmation screen, the chatbot queries the knowledge base with the user's full description. If matching articles exist, they are presented as clickable chips. This reduces unnecessary ticket submissions by surfacing self-service solutions proactively.

7.4 Knowledge Base Module

The Knowledge Base (KB) is a curated repository of IT solutions accessible to all authenticated users.

Features:

- Full-text search via MongoDB's `$text` index on `title` and `content` fields
- Category filter (Hardware, Software, Network, Access, etc.)
- View count — incremented on every `GET /api/kb/:id` call
- **Helpful/Not Helpful ratings** — users vote with thumb up/down; counters stored on the article document
- **Staff inline editing** — agents and admins click a pencil icon to edit the title, content (Markdown), category, tags, and published status in-place without navigating away
- Markdown rendering in the article viewer using escape-first regex substitution

---

7.5 Admin Dashboard & Analytics

The Admin Dashboard (`/admin`) is the primary workspace for agents and administrators.

Three tabs:

Overview tab:

- KPI cards: Total tickets, Open, In Progress, Resolved, Closed — with trend indicators
- Date range filter: 7 days / 30 days / 90 days / All time
- 30-day ticket volume line chart (Recharts `LineChart`)
- Category distribution bar chart (Recharts `BarChart`)
- Agent leaderboard — top agents by resolved ticket count
- Full ticket table with search, filter, sort, bulk status update, and bulk delete

SLA Breach tab:

Computes tickets where elapsed time since creation exceeds the priority SLA threshold. The breach list is sorted oldest-first for triage priority. Columns: Ticket ID, Title, Priority, Status, Assignee, Age (hours), SLA Limit, Overdue By.

Ticket Aging tab:

Segments all open/in-progress tickets into 4 age buckets: Under 1 day, 1–3 days, 3–7 days, Over 7 days. Both a bucket summary card row and a full sorted table are shown.

7.6 User & Role Management

Admins access `/admin/users` for full user lifecycle management:

- Paginated user table with client-side search by name or email
- Per-user: name, email, role badge, active/inactive status, 2FA indicator
- Role dropdown (user → agent → admin) — saved immediately via `PATCH /api/users/:id`
- isActive toggle — deactivated users receive 403 on next login attempt

---

7.7 Email Notification System

All email is sent via the **Gmail REST API** (not SMTP), using `googleapis` npm package with OAuth2 credentials. This was a deliberate engineering decision because Render's free tier blocks outbound SMTP ports 465 and 587; the Gmail REST API uses HTTPS port 443, which is never blocked.

All email uses the `wrap(title, body)` HTML template:

- Dark-themed responsive layout (600px max-width)
- Gradient header with HiTicket branding
- Priority chips: color-coded by severity (Critical=red, High=orange, Medium=blue, Low=green)
- Status chips: color-coded by state

Email events:

Event	Trigger	Function
Ticket created	POST /api/tickets	`sendTicketCreated`
Status changed	PATCH /:id (status)	`sendStatusChanged`
Comment added	PATCH /:id (comment)	`sendCommentAdded`
Ticket assigned	Ticket creation / reassignment	`sendTicketAssigned`
Ticket escalated	Manual escalation	`sendTicketEscalated`
Password changed	POST /users/change-password	`sendPasswordChanged` (always — security alert)
OTP code	2FA login / setup	`sendOTPEmail` (always — security flow)
Weekly digest	Monday 08:00 UTC cron	`sendWeeklyDigest`

All `send*` calls are made **outside the request-response cycle** using `.catch(() => {})`. Email failures never cause API failures.

Weekly Digest (`node-cron`):

Schedule: `0 8 1` (every Monday at 08:00 UTC). For each active user with tickets created in the last 7 days, a personalized digest email is sent showing: total tickets, resolved, open, and up to 5 recent ticket titles with statuses. Only sent if `user.notificationPrefs.weeklyDigest === true`.

7.8 File Attachment System

File uploads use `multer` with `memoryStorage` — files are NEVER written to the server's disk (which is ephemeral on Render). File buffers are streamed directly to Cloudinary via `upload_stream`.

Upload flow:

1. Frontend sends `multipart/form-data` with up to 5 files
2. `multer.array('files', 5)` captures raw buffers in `req.files`

3. `Promise.all(req.files.map(f => uploadToCloud(f.buffer, options)))` — concurrent uploads

4. Cloudinary's CDN URLs are stored in `ticket.attachments[]` sub-documents

5. Deletion uses `cloudinary.uploader.destroy(publicId)` — referenced by Cloudinary public ID, not URL

Security: Filenames are sanitized: `originalname.replace(/[^a-zA-Z0-9._-]/g, '_')`. Files are served over Cloudinary's HTTPS CDN — never directly from the application server.

7.9 Progressive Web App (PWA)

Configuration (`vite.config.js`):

```
VitePWA({
  registerType: 'autoUpdate',
  manifest: {
    name: 'HiTicket',
    short_name: 'HiTicket',
    theme_color: '#7c3aed',
    display: 'standalone',
    icons: [{ src: '/icon-192.png', sizes: '192x192', type: 'image/png' }],
  },
  workbox: {
    runtimeCaching: [{
      urlPattern: /\/api\//,
      handler: 'NetworkFirst',
      options: {
        cacheName: 'api-cache',
        networkTimeoutSeconds: 10,
        expiration: { maxEntries: 50, maxAgeSeconds: 300 },
      },
    }],
  },
})
```

NetworkFirst strategy for API routes: the service worker always tries the network first (10-second timeout), falling back to cached data (up to 5 minutes old, max 50 entries) when offline. Static assets (JS, CSS, images) are pre-cached by Workbox on service worker installation.

Users on Chrome desktop and Android receive an install prompt; iOS users can use "Add to Home Screen" via Safari Share menu.

7.10 UI / Design System

The entire UI is built in **Tailwind CSS v3** without any external component library (no MUI, Ant Design, Chakra UI). Every component — cards, badges, modals, tables, stats, breadcrumbs, inputs — is purpose-built.

Colour tokens:

Token	Hex	Usage
Accent blue	`#3b82f6`	CTA buttons, links, KPI tiles, gradient headers
Page background	`#09090b`	Deepest background
Card surface	`#18181b`	Cards, panels, modals
Input / alt cards	`#111113`	Form inputs, landing cards
Elevated surface	`#27272a`	Table rows, hover states
Border	`#27272a` / `#3f3f46`	Dividers and hover borders
Text primary	`#fafafa`	Headings, labels
Text secondary	`#a1a1aa`	Body, captions
Green	`#22c55e`	Open tickets, success
Amber	`#f59e0b`	In Progress, warnings
Red	`#ef4444`	Critical, SLA breach, delete
Purple	`#8b5cf6`	Admin actions, KPI tile

Inner page structure (standardized across all pages):

```
PageWrapper → Breadcrumb → Gradient Header → Full-width content
bg-gradient-to-r from-[#3b82f6]/8 via-[#6366f1]/4 to-transparent
```

UX Features:

- `⌘K` / `Ctrl+K` — Command Palette for instant navigation
- `?` key — Keyboard Shortcuts reference modal (3 sections: Navigation, Chatbot, General)
- Onboarding Tour — 5-step animated modal shown once on first login (localStorage-gated)
- Copy Ticket ID — clipboard API with 1.5s visual checkmark feedback
- Confetti animation — CSS particle burst on ticket resolved
- Skeleton loaders — shimmer placeholders during API fetches
- Toast notifications — portal-rendered, stacked, 3-second auto-dismiss

---

Chapter 8: Security Implementation

8.1 Authentication Security

Mechanism	Implementation	Protects Against
Password hashing	bcrypt with high cost factor (intentionally slow, resistant to GPU attacks)	Brute-force dictionary attacks, rainbow tables
JWT HS256	Strong secret, long-lived with server-enforced expiry	Forgery without the secret key
Token versioning	`tokenVersion` incremented on login/logout/password-change	Session hijacking — old tokens instantly invalidated
2FA TOTP	speakeasy RFC 6238, clock drift tolerance applied	Account takeover even if password is compromised
2FA Email OTP	Time-limited numeric OTP, `select: false` in schema	Account takeover; OTP never exposed via API
Temp 2FA token	Typed token rejected by `protect` middleware	Unauthorized API access during 2FA verification step

---

8.2 Authorization & Access Control

Every API route is protected server-side. Three middleware levels:

```
protect       // Valid JWT + matching tokenVersion → sets req.user
agentOrAdmin  // protect + role is 'agent' or 'admin'
adminOnly     // protect + role is exactly 'admin'
```

Ownership check on ticket routes:

```
if (!isStaff && ticket.createdBy.toString() !== req.user._id.toString())
  return res.status(403).json({ error: 'Access denied' });
```

Internal notes are stripped from API responses for non-staff users:

```
if (!isStaff) {
  const t = ticket.toObject();
  delete t.internalNotes;
}
```

8.3 OWASP Top 10 Mapping

OWASP Risk	Mitigation
A01 — Broken Access Control	`protect` + `agentOrAdmin` + `adminOnly` middleware on every route. Ownership checks. Internal notes stripped from user responses.
A02 — Cryptographic Failures	bcrypt cost 12. JWT HS256. TOTP secrets and OTPs: `select: false` — never returned in API. MongoDB Atlas TLS connection. HTTPS enforced by Vercel/Render.
A03 — Injection (NoSQL)	`express-mongo-sanitize` strips `$` and `.` from all `req.body`, `req.query`, `req.params` on every request.
A03 — Injection (XSS)	Helmet sets `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `X-XSS-Protection`. ChatBubble uses escape-first Markdown rendering.
A05 — Security Misconfiguration	Helmet (12 headers). CORS strict allowlist — exact origins only, no wildcard subdomains. JSON body limit 50kb. `trust proxy: 1` for accurate rate-limit IPs.
A07 — Identification & Auth Failures	Strict rate limiting on auth endpoints (failed attempts only). Token versioning. Inactivity auto-logout on frontend. Pre-auth temp tokens rejected for API access.
A08 — Software & Data Integrity	`package-lock.json` locks dependency versions. `.npmrc` `legacy-peer-deps` only for known compat issue.
A10 — SSRF	No user-supplied URLs fetched server-side. Cloudinary upload uses in-memory buffer, never a URL from user input.

8.4 Rate Limiting

Limiter	Applied To	Policy
`globalLimiter`	All routes	IP-based threshold per rolling window
`authLimiter`	`/register`, `/login`, `/2fa/verify`	Strict IP-based limit (failed attempts only) per rolling window
`otpLimiter`	`/2fa/resend`, `/2fa/setup/*`	Strict IP-based limit per rolling window

8.5 Data Sanitization

`express-mongo-sanitize` is applied globally before any route handler. It recursively removes all keys starting with `$` and all keys containing `.` from request body, query string, and route parameters. This prevents NoSQL injection attacks of the form:

```
// Malicious body without sanitization:
{ "email": { "$gt": "" }, "password": { "$gt": "" } }
// → Would log in as any user without knowing the password
// With mongo-sanitize: the $gt keys are stripped, query fails safely
```

Chapter 9: Testing

9.1 Testing Approach

Testing was conducted as a combination of **manual functional testing** (feature-by-feature verification against requirements) and **build-time validation** (Vite's TypeScript-compatible build process catches import errors and dead references at compile time).

9.2 Functional Test Cases

Test ID	Module	Test Case	Expected Result	Status
TC-01	Auth	Register with valid credentials	Account created, JWT returned, user redirected to dashboard	✅ PASS
TC-02	Auth	Login with incorrect password	HTTP 401 returned, error message displayed	✅ PASS
TC-03	Auth	Login with 2FA (Email OTP)	OTP sent, verification screen shown, full JWT after correct code	✅ PASS
TC-04	Auth	Login with 2FA (TOTP)	TOTP code verified by speakeasy with window:2, full JWT issued	✅ PASS
TC-05	Auth	Access protected route without token	HTTP 401 returned, redirected to login	✅ PASS
TC-06	Auth	Access admin route as regular user	HTTP 403 returned	✅ PASS
TC-07	Ticket	Create ticket via chatbot	Ticket created with auto-assigned ticketId (TKT-NNNN)	✅ PASS
TC-08	Ticket	Auto-assignment to least-loaded agent	Ticket assigned to agent with fewest open tickets	✅ PASS
TC-09	Ticket	Update status from Open to In Progress (agent)	Status updated, history entry added, email sent	✅ PASS
TC-10	Ticket	User attempts to set status to Closed (disallowed)	HTTP 403, "users can only reopen" error	✅ PASS
TC-11	Ticket	Upload file attachment (Cloudinary)	File stored on CDN, URL saved in ticket.attachments	✅ PASS
TC-12	Ticket	Watch / unwatch ticket	Watchers array updated, watcher count correct	✅ PASS

TC-13	KB	Search knowledge base by keyword	Relevant articles returned via $text index	✅ PASS
TC-14	KB	KB deflection in chatbot step 3	Matching articles appear before confirmation	✅ PASS
TC-15	KB	Helpful / Not Helpful vote	Counter incremented in kbarticles document	✅ PASS
TC-16	Admin	Dashboard KPI counts	Counts match direct MongoDB queries	✅ PASS
TC-17	Admin	SLA Breach tab	Tickets older than SLA threshold displayed	✅ PASS
TC-18	Admin	Ticket Aging tab	Tickets correctly bucketed by age	✅ PASS
TC-19	Admin	Bulk delete tickets	Selected tickets removed, operation confirmed	✅ PASS
TC-20	Email	sendTicketCreated fires on creation	Email delivered to submitter via Gmail API	✅ PASS
TC-21	Email	sendPasswordChanged fires after change	Security notification email received	✅ PASS
TC-22	Security	NoSQL injection attempt in login	`$gt` key stripped by mongo-sanitize, attack fails	✅ PASS
TC-23	Security	Auth endpoint brute-force	Blocked after 10 failed attempts, 429 returned	✅ PASS
TC-24	PWA	Offline access	Cached API responses served by service worker	✅ PASS
TC-25	PWA	Install prompt	Browser shows install button on supported platforms	✅ PASS

9.3 Security Test Scenarios

Scenario	Method	Result
JWT from logged-out session used	Old token re-used after logout (tokenVersion changed)	HTTP 401 — token invalidated ✅
2FA temp token used as session token	`tempToken` submitted to `/api/tickets`	HTTP 401 — pre-auth token rejected by `protect` middleware ✅
IDOR — access other user's ticket	GET `/api/tickets/` as regular user	HTTP 403 — ownership check blocks ✅
Internal notes in API response	GET full ticket as regular user	`internalNotes` array absent from response ✅
Direct NoSQL injection in login body	`{ "email": { "$gt": "" } }`	mongo-sanitize strips `$gt`, login fails safely ✅

9.4 Build Validation

Every code change was validated by running `npx vite build` before committing to the main branch. Vite's build process:

- Resolves all ES module imports — dead references cause build failure
- Tree-shakes unused code
- Produces a `dist/` folder with hashed filenames for CDN caching
- Reports bundle size warnings (chunk size warning at 500kb+) — informational only

Final production build: **CSS 76.23 kB (gzip ~12 kB), JS 1,191.86 kB (gzip ~340 kB)** — built in approximately 900ms.

Chapter 10: Deployment

10.1 Deployment Architecture

```
GitHub (Monorepo: /IT Ticketing/)
        │
        ├── helpdesk-ai/   ──→   Vercel Project
        │                         Build: npm run build (Vite)
        │                         Output: dist/
        │                         CDN: Global edge network
        │
        └── helpdesk-api/  ──→   Render Web Service
                                   Start: node server.js
                                   Node.js 20
                                   Auto-deploy on push to main
```

Both Vercel and Render support **subdirectory deployments** — each service is pointed at its respective subdirectory root (`helpdesk-ai/` and `helpdesk-api/`).

10.2 Frontend — Vercel

- **Build command:** `npm run build` (Vite)
- **Output directory:** `dist/`
- **Install command:** `npm install` (uses `.npmrc` with `legacy-peer-deps=true`)
- **Environment variable:** `VITE_API_URL=https://.onrender.com/api`
- **Auto-deploy:** Triggered on every push to `main` branch
- **CDN:** Vercel's global edge network serves the static SPA from the nearest PoP to each user

Why `.npmrc legacy-peer-deps=true`?

`vite-plugin-pwa@1.2.0` declares a peer dependency on `vite@^7`, but the project uses `vite@8.0.0`. Without this flag, Vercel's `npm install` throws `ERESOLVE` and the build fails. The flag instructs npm to use the legacy peer dependency resolution algorithm (npm v6 behavior) which allows the version mismatch.

10.3 Backend — Render

- **Service type:** Web Service
- **Build command:** `npm install`
- **Start command:** `node server.js`
- **Node.js version:** 20
- **Auto-deploy:** Triggered on every push to `main` branch
- **Port:** Render auto-assigns and exposes via `$PORT` environment variable

Cold start note: Render's free tier spins down instances after 15 minutes of inactivity. The first request after a spin-down takes approximately 25–40 seconds while the instance cold-starts. Subsequent requests are fast.

10.4 Database — MongoDB Atlas

- **Tier:** Shared M0 (free tier)
- **Cluster:** Multi-region replication
- **Network access:** IP allow-list includes `0.0.0.0/0` (Render free tier does not provide static IPs — a known limitation; upgrading to a paid Render tier with static IP egress would allow this to be locked down)
- **Connection:** `MONGO_URI` with TLS — Mongoose connects with `mongoose.connect(MONGO_URI)`

10.5 Environment Variables

Backend (`helpdesk-api/.env`):

Variable	Description
`MONGO_URI`	MongoDB Atlas TLS connection string
`JWT_SECRET`	HS256 signing secret (cryptographically random, minimum 256-bit entropy)
`PORT`	Auto-set by Render
`CLIENT_URL`	Frontend origin for CORS (`https://hiticket.vercel.app`)
`EMAIL_USER`	Gmail sender address
`GMAIL_CLIENT_ID`	Google Cloud OAuth2 client ID
`GMAIL_CLIENT_SECRET`	Google Cloud OAuth2 client secret
`GMAIL_REFRESH_TOKEN`	Long-lived refresh token from OAuth Playground
`CLOUDINARY_CLOUD_NAME`	Cloudinary cloud identifier
`CLOUDINARY_API_KEY`	Cloudinary API key
`CLOUDINARY_API_SECRET`	Cloudinary API secret

Frontend (`helpdesk-ai/.env`):

Variable	Description
`VITE_API_URL`	Backend API base URL (`https://.onrender.com/api`)

10.6 CI/CD Pipeline

A lightweight CI/CD pipeline is implemented through Git-triggered auto-deploys:

1. Developer pushes code to `main` branch on GitHub
2. Vercel detects the push — builds and deploys the frontend automatically (~1 min)
3. Render detects the push — installs dependencies, restarts the Node.js process (~2 min)

4. Both deployments operate independently — a failing frontend deploy does not affect the backend

Pre-deployment validation (developer-enforced):

- Run `npx vite build` locally — ensures zero build errors before push
- Manually verify critical flows (login, ticket creation, email) on the live URL after deploy

---

Chapter 11: Results & Discussion

11.1 Achieved Outcomes

All 24 functional requirements defined in Chapter 3 have been implemented and validated. The following table summarizes the delivery status:

Requirement Area	Status
User registration and login	✅ Complete
Two-factor authentication (TOTP + Email OTP)	✅ Complete
JWT session management with token versioning	✅ Complete
4-step chatbot ticket creation wizard	✅ Complete
Auto-categorization and priority detection	✅ Complete
Round-robin agent auto-assignment	✅ Complete
Knowledge base with search, ratings, staff editing	✅ Complete
KB deflection in ticket creation flow	✅ Complete
Admin dashboard (KPIs, charts, leaderboard)	✅ Complete
SLA Breach tracking tab	✅ Complete
Ticket Aging analysis tab	✅ Complete
Role-based access control (3 roles)	✅ Complete
User management (activate, deactivate, role change)	✅ Complete
Internal notes (staff-only, never exposed)	✅ Complete
Audit history on every ticket	✅ Complete
Email notifications (8 event types)	✅ Complete
Weekly digest cron (Mondays 08:00 UTC)	✅ Complete
File attachments via Cloudinary	✅ Complete
CSAT post-resolution survey	✅ Complete
Ticket watcher feature	✅ Complete
Due date assignment	✅ Complete
Progressive Web App (installable, offline)	✅ Complete
Activity log (admin audit trail)	✅ Complete

Bulk ticket operations (status update, delete)	✅ Complete

11.2 UI Walkthrough

Home Page (unauthenticated):

The landing page features an ambient-glow hero section with a gradient headline, social proof, and a ticket creation mockup card. Key platform statistics are displayed in a divide-x KPI strip (2,400+ resolved, 98% satisfaction, <2h resolution, 8s creation). Subsequent sections cover How It Works (4-col card grid), Features (bento grid layout), Use Case roles, Tech Stack badges, Survey CTA, and a gradient Final Call-to-Action.

Authenticated Dashboard:

After login, the Home page transforms into a command center with a 6-tile KPI ribbon (Open, In Progress, Resolved, Critical, Resolution%, Total), a ticket timeline with recent activity, a donut-chart overview with status bars, priority breakdown, top categories, and quick navigation tiles to Chatbot, Tickets, Calendar, and Reports.

Chatbot:

A conversational step-by-step interface. Step 1 shows a category tile grid. Step 2 renders context-sensitive sub-type chips. Step 3 shows form fields with live KB suggestions. Step 4 displays the ticket summary for confirmation.

Admin Dashboard:

Three-tab layout. Overview shows live stats and Recharts visualizations. SLA Breach shows a red-themed table of overdue tickets. Ticket Aging shows bucket cards + a full table sorted oldest-first.

11.3 Performance Observations

Metric	Observation
Frontend build time	~900ms (Vite, production build)
API response times (Render, warm)	150–400ms typical for complex queries
API cold start (Render free tier)	25–40 seconds after 15-min inactivity
Bundle size	JS: 1,191 kB raw / ~340 kB gzip; CSS: 76 kB
PWA install size	~350 kB (compressed) after Workbox precache
KB search latency	<100ms with MongoDB `$text` index

The Render free-tier cold start is the primary UX limitation. This is an infrastructure constraint of the free hosting tier, not an application design issue. Upgrading to Render's paid tier ($7/month) eliminates cold starts with always-on instances.

Chapter 12: Conclusion & Future Scope

12.1 Conclusion

HiTicket demonstrates a complete, production-deployed full-stack web application that addresses a real organizational problem — unstructured IT support management. The system successfully implements all planned features across authentication, ticket lifecycle management, analytics, knowledge base, email notifications, file storage, and PWA capabilities.

Key engineering achievements of the project include:

1. **Clean three-tier architecture** — decoupled frontend (Vercel) and backend (Render) communicating via a well-defined REST API, with MongoDB Atlas as the data tier.

2. **Enterprise-grade security** — JWT with token versioning, bcrypt password hashing, two-factor authentication (both TOTP and Email OTP), OWASP Top 10 mitigations, and three-level RBAC.

3. **Intelligent ticket creation** — state-machine chatbot with keyword-based auto-categorization, priority detection, KB deflection, and 6 pre-built templates — all without an LLM dependency.

4. **Production reliability** — Vite build validation, Gmail REST API for email (bypassing SMTP port blocks), Cloudinary CDN for files (bypassing Render's ephemeral disk), and node-cron for scheduled tasks.

5. **Polished user experience** — consistent dark design system, command palette, keyboard shortcuts, PWA support, skeleton loaders, toast notifications, and responsive mobile layout.

The project demonstrates practical mastery of full-stack web development, cloud deployment, RESTful API design, secure authentication architecture, NoSQL data modelling, and modern frontend engineering practices.

12.2 Future Enhancements

Feature	Priority	Description
Real-time notifications (WebSocket/SSE)	High	Replace polling with Socket.io or Server-Sent Events for live comment updates and ticket status changes
SSO (Google / Microsoft OAuth)	Medium	passport.js OAuth2 strategy; allow login with existing organizational Google/Microsoft accounts
AI Priority Prediction	Medium	Replace keyword regex with a trained text classification model (OpenAI API or fine-tuned BERT) for more accurate categorization and priority assignment
Multi-language / i18n	Medium	`react-i18next` integration for regional language support
Canned Responses	Medium	Agent-side dropdown of pre-written reply templates, stored in localStorage and customizable
SLA Auto-Escalation	High	Automatically escalate ticket priority or reassign when SLA threshold is breached
Native Mobile App	Low	React Native app sharing the existing Node.js backend API
Multi-tenant / Department Isolation	Low	Workspace scoping per department — separate ticket queues, KB articles, and user namespaces
IP Session Management	Low	Admin ability to view login IPs and remotely revoke specific sessions

---

Chapter 13: References

1. **React Documentation** — Meta Open Source. *React – The library for web and native user interfaces.* https://react.dev

2. **Express.js Documentation** — OpenJS Foundation. *Express — Fast, unopinionated, minimalist web framework for Node.js.* https://expressjs.com

3. **MongoDB Documentation** — MongoDB Inc. *MongoDB Manual.* https://www.mongodb.com/docs/manual/

4. **Mongoose Documentation** — Automattic. *Mongoose — Elegant MongoDB object modeling for Node.js.* https://mongoosejs.com/docs/

5. **JSON Web Tokens** — Auth0. *Introduction to JSON Web Tokens.* https://jwt.io/introduction/

6. **RFC 6238** — TOTP: Time-Based One-Time Password Algorithm. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc6238

7. **OWASP Top 10** — OWASP Foundation. *OWASP Top Ten Web Application Security Risks (2021).* https://owasp.org/Top10/

8. **bcryptjs** — Dani Protsenko. *bcryptjs — Optimized bcrypt in JavaScript with zero dependencies.* https://github.com/dcodeIO/bcrypt.js

9. **Tailwind CSS Documentation** — Tailwind Labs Inc. *Tailwind CSS — Rapidly build modern websites without ever leaving your HTML.* https://tailwindcss.com/docs

10. **Vite Documentation** — Evan You & contributors. *Vite — Next Generation Frontend Tooling.* https://vitejs.dev/guide/

11. **Cloudinary Documentation** — Cloudinary Ltd. *Cloudinary Developer Documentation.* https://cloudinary.com/documentation

12. **Google Gmail API** — Google LLC. *Gmail API Reference.* https://developers.google.com/gmail/api

13. **Helmet.js** — Adam Baldwin & contributors. *Helmet — Help secure Express apps with various HTTP headers.* https://helmetjs.github.io/

14. **node-cron** — Lucas Merencia. *node-cron — A simple cron-like job scheduler for Node.js.* https://github.com/node-cron/node-cron

15. **vite-plugin-pwa** — Anthony Fu & contributors. *vite-plugin-pwa — Zero-config PWA for Vite.* https://vite-pwa-org.netlify.app/

16. **Recharts** — recharts group. *A composable charting library built on React components.* https://recharts.org

17. **Workbox** — Google LLC. *Workbox — JavaScript libraries for adding offline support to web apps.* https://developer.chrome.com/docs/workbox

18. **speakeasy** — Mark Bao & contributors. *speakeasy — Two-factor authentication for Node.js.* https://github.com/speakeasyjs/speakeasy

19. **MERN Stack Architecture** — Joanna Smith. *Modern Web Application Development with the MERN Stack.* FreeCodeCamp, 2023. https://www.freecodecamp.org

20. **REST API Design Best Practices** — Martin Fowler. *Richardson Maturity Model.* https://martinfowler.com/articles/richardsonMaturityModel.html

---

Appendix

Appendix A — API Endpoint Reference

Auth Routes (`/api/auth`)

Method	Endpoint	Auth Required	Description
POST	`/register`	No	Register new user (rate-limited)
POST	`/login`	No	Login; returns JWT or 2FA challenge
POST	`/2fa/verify`	No (uses tempToken)	Complete 2FA; returns full JWT
POST	`/2fa/resend`	No (uses tempToken)	Resend email OTP
POST	`/2fa/setup/email`	Yes	Begin email 2FA enrollment
POST	`/2fa/setup/email/verify`	Yes	Confirm email 2FA enrollment
POST	`/2fa/setup/totp`	Yes	Generate TOTP secret + QR code
POST	`/2fa/setup/totp/verify`	Yes	Confirm TOTP code; enable 2FA
POST	`/2fa/disable`	Yes	Disable 2FA (requires current code)

Ticket Routes (`/api/tickets`)

Method	Endpoint	Auth	Description
GET	`/public/:ticketId`	None	Public ticket status lookup
GET	`/`	Any authenticated	List tickets (own for users / all for staff)
GET	`/stats`	Agent+Admin	KPI counts by status
GET	`/:id`	Any authenticated	Full ticket (internalNotes stripped for users)
POST	`/`	Any authenticated	Create ticket (auto-assigns agent)
PATCH	`/:id`	Any authenticated	Update status, comment, CSAT
POST	`/:id/notes`	Agent+Admin	Add internal note
POST	`/:id/attachments`	Any authenticated	Upload up to 5 files
DELETE	`/:id/attachments/:aid`	Any authenticated	Remove attachment
PATCH	`/bulk`	Agent+Admin	Bulk status update
DELETE	`/bulk`	Admin only	Bulk delete
DELETE	`/:id`	Admin only	Hard delete single ticket
PATCH	`/:id/watch`	Any authenticated	Toggle watcher
PATCH	`/:id/due-date`	Agent+Admin	Set or clear due date

User Routes (`/api/users`)

Method	Endpoint	Auth	Description
GET	`/profile`	Any authenticated	Get own profile
PUT	`/profile`	Any authenticated	Update profile fields
PUT	`/notifications`	Any authenticated	Update notification preferences
POST	`/change-password`	Any authenticated	Change password (validates current)
GET	`/`	Admin only	List all users
PATCH	`/:id`	Admin only	Update role or isActive

Knowledge Base Routes (`/api/kb`)

Method	Endpoint	Auth	Description
GET	`/`	Any authenticated	List articles (supports `?q` , `?category`)
GET	`/:id`	Any authenticated	Single article (increments views)
POST	`/`	Agent+Admin	Create article
PUT	`/:id`	Agent+Admin	Update article
DELETE	`/:id`	Agent+Admin	Delete article
POST	`/:id/helpful`	Any authenticated	Vote helpful / not helpful

Other Routes

Method	Endpoint	Auth	Description
GET	`/api/logs`	Admin only	Activity audit log
POST	`/api/feedback`	None	Submit post-resolution survey
GET	`/api/feedback`	Admin only	List all feedback
GET	`/api/health`	None	`{ status: 'ok', time }`

Appendix B — Keyboard Shortcuts

Shortcut	Action
`⌘K` / `Ctrl+K`	Open command palette
`⌘N` / `Ctrl+N`	Open new ticket (chatbot)
`G` then `H`	Go to Home
`G` then `T`	Go to My Tickets
`G` then `K`	Go to Knowledge Base
`G` then `P`	Go to Profile
`?`	Open keyboard shortcuts reference modal
`Esc`	Close any open modal
`Tab`	Navigate interactive elements
`Enter`	Confirm / submit focused action

Appendix C — Chatbot Slash Commands

Command	Effect
`/status`	Inline ticket status lookup — prompts for ticket ID (e.g. TKT-0042)
`/agent`	Display agent handoff message and contact information
`/template`	Display 6 pre-built ticket template tiles to choose from
`/kb`	Prompt for a search term and display matching KB articles inline
`/clear`	Reset the entire conversation back to the initial step
`Esc`	Cancel current slash command

HiTicket — AI-Assisted IT Helpdesk & Ticketing Platform

Full-Stack Web Application Project Report

March 2026

Live Application: https://hiticket.vercel.app